

ПРИВАТНИЙ ВИЩИЙ НАВЧАЛЬНИЙ ЗАКЛАД
"ЄВРОПЕЙСЬКИЙ УНІВЕРСИТЕТ"
ФАКУЛЬТЕТУ ІНФОРМАЦІЙНИХ СИСТЕМ ТА ТЕХНОЛОГІЙ
Кафедра МДІП



ЗВІТ

про виконання фінального завдання
з дисципліни "Системний аналіз та проектування
інформаційних систем "
на тему:
«Проектування інформаційної системи обліку робочого
часу».

Виконав: студент
групи:
241\он
Прізвище виконавця:
Петров Павло

Прийняв:
Рихальський О.Ю.

Київ – 2026

Мета роботи

Метою роботи є виконання системного аналізу та проєктування інформаційної системи обліку робочого часу, яка забезпечить структурований збір даних про присутність співробітників, підтримує рольову модель доступу та інтегрується з існуючою певною інфраструктурою компанії.

Завдання

Завданням роботи є продемонструвати здатність спроектувати інформаційну систему повного циклу - від аналізу вимог і моделювання бізнес-процесів до побудови архітектури, інтеграції компонентів і документування результатів.

Інформаційна система, що проєктується в межах фінального завдання, - інформаційна система обліку робочого часу працівників (Time Tracking System), призначена для автоматизації фіксації фактично відпрацьованих годин, перерв, запізнь та інших відхилень від графіка. Її впровадження є актуальним для організацій зі змінною роботою (наприклад, магазини, склади, сервісні компанії), де ручний облік у таблицях або повідомленнях призводить до помилок, втрати даних і складності перевірок.

Метою системи є забезпечення прозорого та контрольованого процесу обліку часу: від реєстрації "прихід/вихід" до формування табелів і підсумкових звітів для управлінських рішень.

Очікуваними користувачами системи є працівники (виконують відмітки часу та переглядають власні підсумки), керівники змін/підрозділів.

Розділ 1.

Обрана ІС:

Проект присвячений розробці інформаційної системи обліку робочого часу для умовної продуктової компанії Farry, що спеціалізується на, скажімо, продажу електроніки.

Система автоматизує фіксацію присутності співробітників, реєстрацію початку та завершення робочого дня, облік відпусток і відсутностей, а також формування звітності для HR-відділу, керівництва та бухгалтерії. Рішення орієнтоване не на «тотальний контроль», а на прозоре управління часом, зменшення ручної рутини та зниження кількості помилок під час нарахування заробітної плати.

Об'єктом дослідження є процеси обліку робочого часу та нарахування заробітної плати в організаціях різних форм власності. У сучасних умовах ведення бізнесу ці процеси відіграють ключову роль у системі управління персоналом, адже безпосередньо впливають на фінансову стабільність компанії, рівень мотивації працівників та дотримання норм трудового законодавства.

Розділ 2.

Збір і аналіз вимог.

Опис предметної області та мета системи.

Інформаційна система обліку робочого часу (далі - Система) призначена для фіксації фактично відпрацьованого часу працівників, контролю присутності/відсутності, формування табелів та підготовки даних для розрахунку заробітної плати або управлінської аналітики.

Система є автономним модулем обліку часу та може інтегруватися з іншими ІС підприємствами (наприклад, ERP магазину, Payroll, в даному випадку PhantomGroup(фін. напрямок) через API/обмін файлами.

Межі системи та зовнішні умови.

В межах системи: облік змін/графіків, відмітки приходу/виходу, перерви, запізнення, понаднормові, відпустки/лікарняні (як події), табель та звіти.

Поза межами системи: нарахування зарплати як фінальний розрахунок може виконуватись в окремій системі, а Система лише передає підготовлені дані (години, коди відхилень, періоди).

Зовнішні умови/інтеграції: корпоративний каталог користувачів, зовнішня HR система, та магазинна система для зв'язку зі змінами, біометричні/турнікетні рішення.

Стейкхолдери.

В якості стейкхолдерів в системі будуть наступні:

1. Працівники: хочуть швидко відмічати час і бачити свої години/переробки.
2. Керівники змін/відділів: планують графіки, затверджують коригування.
3. HR: контролює правила обліку, відпустки/відхилення, звірки табелів.
4. Бухгалтерія/Payroll: отримує табелі та підсумки для розрахунків.
5. Адміністратор ІС: керує доступами, інтеграціями, аудитом і довідниками.

В якості **методу збору вимог** сформовані на основі інтерв'ю з ролями (працівник, керівник, HR, адмін ІС, див. User Stories)

Використана специфікація вимог із функціональними та нефункціональними вимогами. Джерело: [<http://well-architected-guide.com/documents/iso-iec-ieee-29148-template/>]

Функціональні вимоги

FR-01 реєстрація робочого часу. Система повинна дозволяти працівнику виконати check-in та check-out, а також фіксувати перерви (start break / end break).

FR-02 автоматичний підрахунок годин. Система повинна обчислювати тривалість зміни, тривалість перерв, чистий робочий час та (за правилами) понаднормові години.

FR-03 графіки та зміни. Система повинна дозволяти керівнику створювати/публікувати графіки змін (плановий початок/кінець, тип зміни, підрозділ).

FR-04 відхилення та причини. Система повинна підтримувати довідник причин/кодів відхилень (запізнення, відсутність, лікарняний, відпустка, службове відрядження) і прив'язку їх до періодів.

FR-05 коригування та погодження. Система повинна дозволяти працівнику/керівнику створювати запит на коригування (наприклад, пропущений check-in) з коментарем, а керівнику - затвердити або відхилити запит із фіксацією рішення.

FR-06 Табелі за період. Система повинна формувати таблиці обліку робочого часу за вибраний період (місяць/тиждень) з підсумками по працівниках і по підрозділах.

FR-07 Звіти й аналітика. Система повинна надавати звіти: запізнення, понаднормові, відсутності, фактичні години vs план, динаміка по періодах.

FR-08 Управління користувачами та ролями. Система повинна підтримувати ролі та права доступу на основі RBAC (призначення прав через ролі, а не вручну кожному користувачу).

FR-09 Аудит подій. Система повинна вести журнал подій (хто/коли створив або змінив записи часу, хто затвердив коригування).

Нефункціональні вимоги

NFR-01 Продуктивність (час відповіді). Для типових операцій (відкриття табеля, check-in) час відповіді має бути прийнятним для інтерактивної роботи користувача.

NFR-02 Надійність і відновлюваність. Система має забезпечувати збереження даних та можливість відновлення після збою (резервне копіювання/відновлення).

NFR-03 Безпека і принцип найменших привілеїв. Доступ до функцій і даних повинен визначатися ролями, щоб користувач мав лише ті права, які потрібні для його роботи. [2]

NFR-04 Аудит і простежуваність. Критичні дії (коригування часу, затвердження, зміни довідників) повинні бути простежуваними через журнал подій.

NFR-05 Сумісність. Система має передбачати інтеграцію з іншими ІС через стандартизовані інтерфейси.

NFR-06 Зручність використання. Інтерфейс повинен мінімізувати кількість кроків для check-in/check-out і не вимагати спеціального навчання для базових операцій.

Розділ 3.

Опис різних категорій користувачів.

Доступи пропонується реалізувати через рольову модель RBAC: права призначаються ролям, а ролі - користувачам.

Групи користувачів

1. **Працівник** (Employee).
2. **Керівник зміни / менеджер підрозділу** (**Manager**).
3. **HR-спеціаліст** (HR).
4. **Адміністратор системи** (System Admin).

Ця система, *(система обліку робочого часу)* буде орієнтована на **чотири основні категорії користувачів**, *(орієнтуючись на компанію PhantomGroup, звідси)* які будуть мати різні рівні доступу до системи, а також різні функціональні можливості;

1. **Працівник звичайний:**

Буде використовувати систему виключно для базових операцій з реєстрації робочого часу, включаючи позначки початку та завершення робочої зміни, перегляд власної статистики присутності, + подання електронних заявок на відпустки.

2. Лінійний керівник(або адмін) отримує розширені інструменти для управління своєю командою, зокрема можливість контролю присутності підлеглих(скасувати зміну, виправити час роботи й т.п), затвердження заявок на відсутність(відпустка), формування звітів по своєму підрозділу та встановлення індивідуальних робочих графіків

для співробітників(формування розкладу роботи).

3. Спеціаліст кадрової служби - буде володіти найширшими повноваженнями в системі, що включають адміністрування облікових записів персоналу(додати, видалити, оновити), налаштування корпоративних календарів та святкових днів, ведення обліку відпусток і лікарняних по всій компанії, + генерування комплексної звітності для бухгалтерії та керівництва вище.

4. Адміністратор технічний - буде забезпечувати підтримку функціонування системи на технічному рівні через управління користувацькими правами доступу, налаштування інтеграцій з корпоративною інфраструктурою, виконання регулярного резервного копіювання та надання технічної підтримки всім категоріям користувачів.

Схематичне представлення на рис. 3.1.



Рис. 3.1

Розділ 4.

Функціональні сценарії (User Stories).

Використаю вже наявні User Stories, але більш конкретизовано. Подано в форматі «роль», яка має «ціль», щоб отримати «вигоду». [3]

(Тобто як acceptance criteria (це набір перевірюваних тверджень, які описують, яким має бути результат реалізації конкретної задачі. Далі - АС.)

US-1 (Працівник): відмітка початку/кінця зміни
Як **працівник**, я хочу відмічати початок і завершення робочої зміни в застосунку (PC або mobile), щоб мати точний облік годин і уникати помилок у табелі та нарахуванні.

АС:

- Система дозволяє виконати "Почати зміну" і "Завершити зміну" з фіксацією дати/часу та пристрою/каналу (PC /mobile).
- Після відмітки користувач бачить підтвердження успішної операції та поточний статус дня (в роботі/завершено).
- Якщо немає зв'язку, система зберігає відмітку локально та синхронізує при відновленні з'єднання (із позначенням факту синхронізації).
- Підтримуються методи ідентифікації (наприклад, QR/штрих-код) як альтернативний спосіб виконати відмітку.

US-2 (Керівник): моніторинг присутності команди
Як **лінійний керівник**, я хочу бачити статус присутності працівників моєї команди в реальному часі, щоб оперативно планувати роботу та реагувати на відхилення від графіка.

АС:

- Система відображає список підлеглих з полями: час приходу, поточний статус (на зміні/на перерві/відсутній), плановий час завершення.
- Є попередження/позначки для критичних відхилень (запізнення, відсутній check-out, неявка на зміну).
- Керівник може ініціювати коригування запису по конкретному працівнику в межах своєї команди, із обов'язковим коментарем причини.
- Усі коригування логуються (хто/коли/що змінив).

US-3 (HR): ведення персоналу та підготовка табелів
Як **HR-спеціаліст**, я хочу керувати даними співробітників і формувати табелі за період з урахуванням свят/відсутностей і понаднормових, щоб зменшити ручну працю та ризик помилок при передачі даних у бухгалтерію/Payroll.

АС:

- HR може додати/оновити/деактивувати профіль співробітника (активний статус впливає на доступ і участь у табелюванні).
- Система підтримує корпоративний календар (святкові/неробочі дні) та застосовує його під час розрахунків.

- Табель формується за обраний період і експортується у визначені формати (мінімум CSV/XLSX).
- Система рахує понаднормові за налаштованими правилами (коефіцієнти/пороги задаються параметрами).

US-4 (Технічний адміністратор): централізовані права доступу та аудит.

Як **технічний адміністратор**, я хочу централізовано керувати ролями та правами доступу користувачів, щоб забезпечити безпеку даних і швидко відображати організаційні зміни.

АС:

- Адміністратор може створювати/редагувати ролі та дозволи (перегляд/редагування/видалення) і призначати ролі користувачам масово.
- При звільненні працівника обліковий запис деактивується, а дані зберігаються в архівному режимі згідно політики компанії.
- Система веде аудит-лог змін із фільтрацією за користувачем та періодом.
- Підтримується резервне копіювання (запуск/контроль виконання з адмін-інтерфейсу або через регламент).

Розділ 5.

User Stories та UML-нотації (User Stories).

Для формування UML діаграм використаю Mermaid Live Editor.

1. Працівник.

5.1 Діаграма відображає стани працівника протягом дня та можливості працівника в системі. Від стартового стану, коли користувач відкриває додаток і проходить реєстрацію приходу, після чого переходить у стан «НаРоботі/Активний». У межах «НаРоботі» передбачені дії/підстани: подання заявки на відпустку, перегляд статистики, перехід на перерву, тощо.

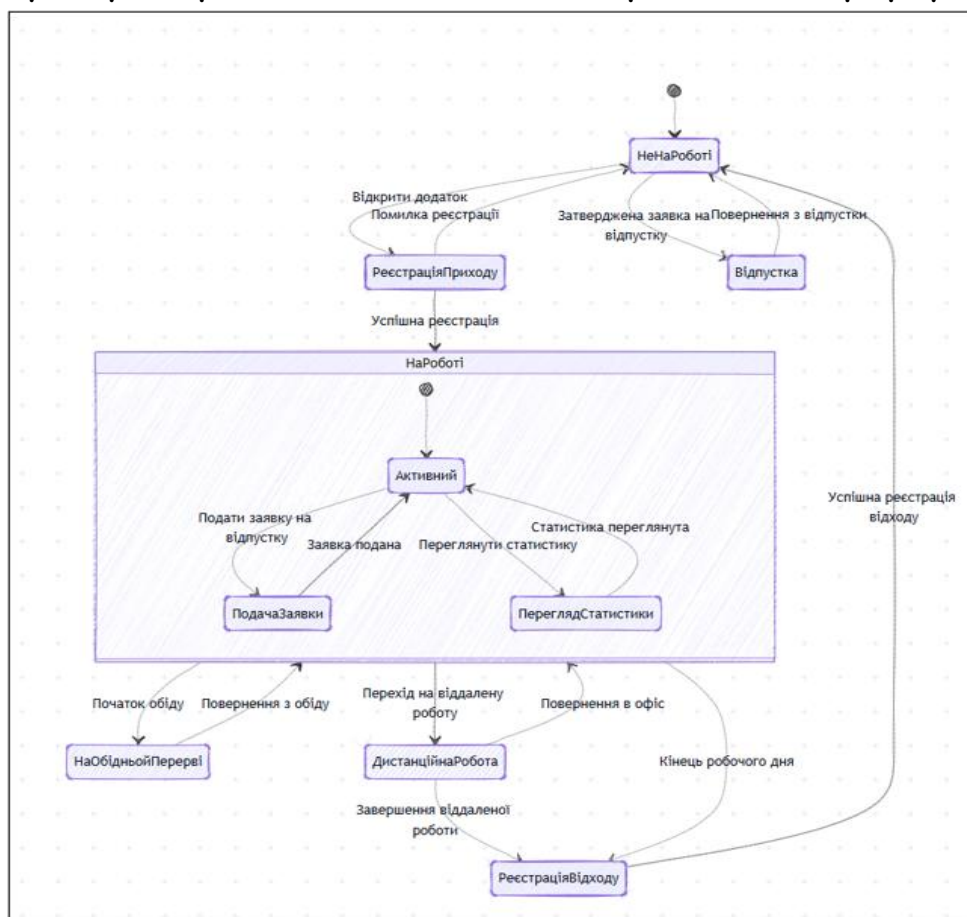


Рис. 5.1. Стани працівника.

5.2 Блок-схема сценарію перегляду статистики працівником у системі обліку робочого часу зобразив наступним чином. Відображено як загально показується сценарій перегляду статистики в системі обліку робочого часу: користувач входить у застосунок, переходить у розділ зі статистикою та обирає період. Після вибору система відображає відповідні показники за період. Також присутній експорт даних.

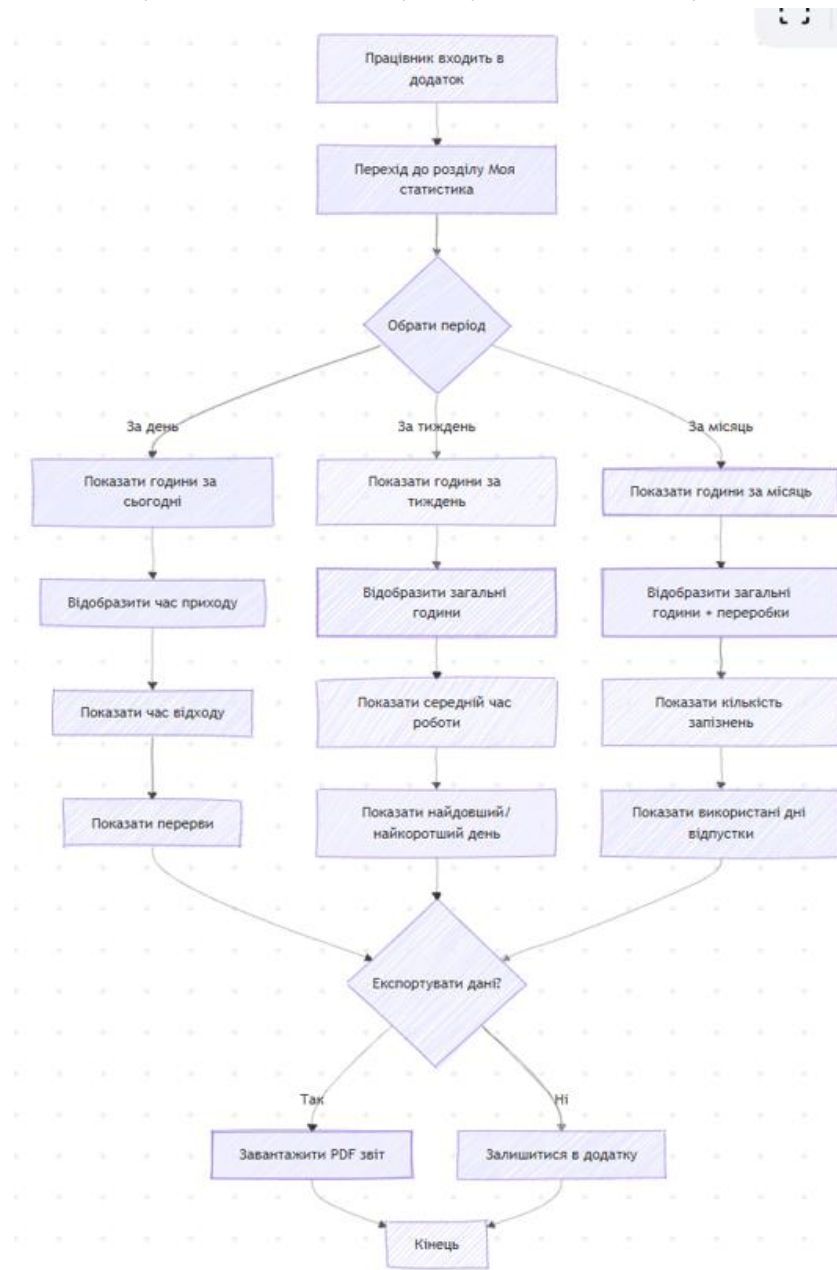


Рис. 5.2 Сценарій перегляду статистики.

5.3 uml-нотація процесу подачі заявки у відпустку працівником. Працівник створює заявку в стані оформлення, після чого заявка переходить на розгляд керівнику. Далі можливі варіанти: керівник затверджує (стан системи стане ready), або відхиляє. Стан sorry - заявка буде закрита.

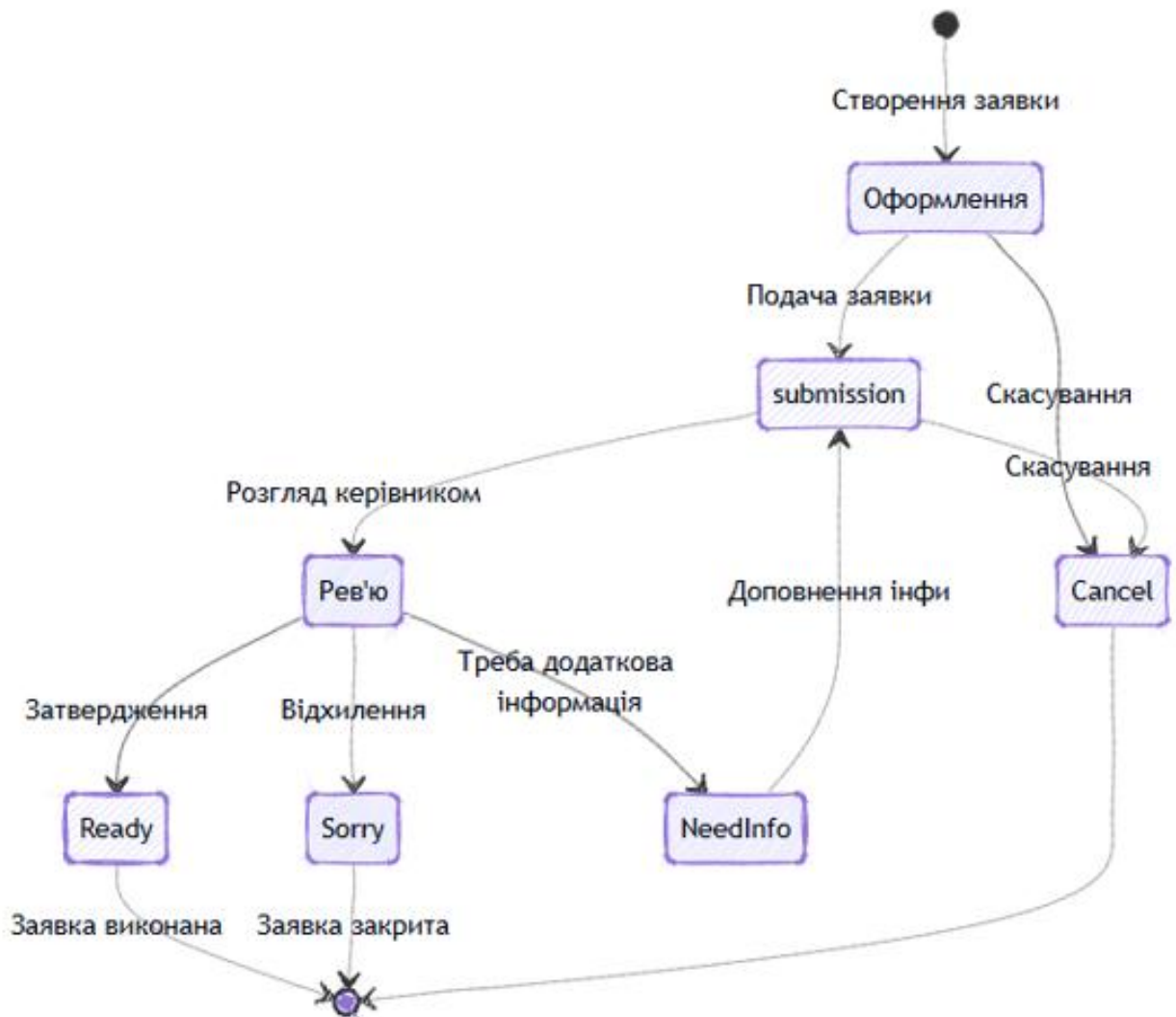


Рис. 5.3 Процес подачі заявки у відпустку.

2. Керівник зміни.

5.4 Функцію лінійного керівника в системі зобразив на мал.

Наглядова панель, де керівник бачить актуальний статус кожного підлеглого (хто на зміні, хто відсутній/на перерві), а також час приходу та плановий час виходу. На основі цього керівник може планувати задачі, координувати взаємодію в команді, виконувати коригування робочого часу/статусів.



Рис. 5.4 Функція лінійного керівника.

3. HR спеціаліст.

Спеціаліст кадрової служби, який буде володіти найширшими повноваженнями в системі, що включають адміністрування облікових записів персоналу.

Функції в системі зображено на рис 5.5:

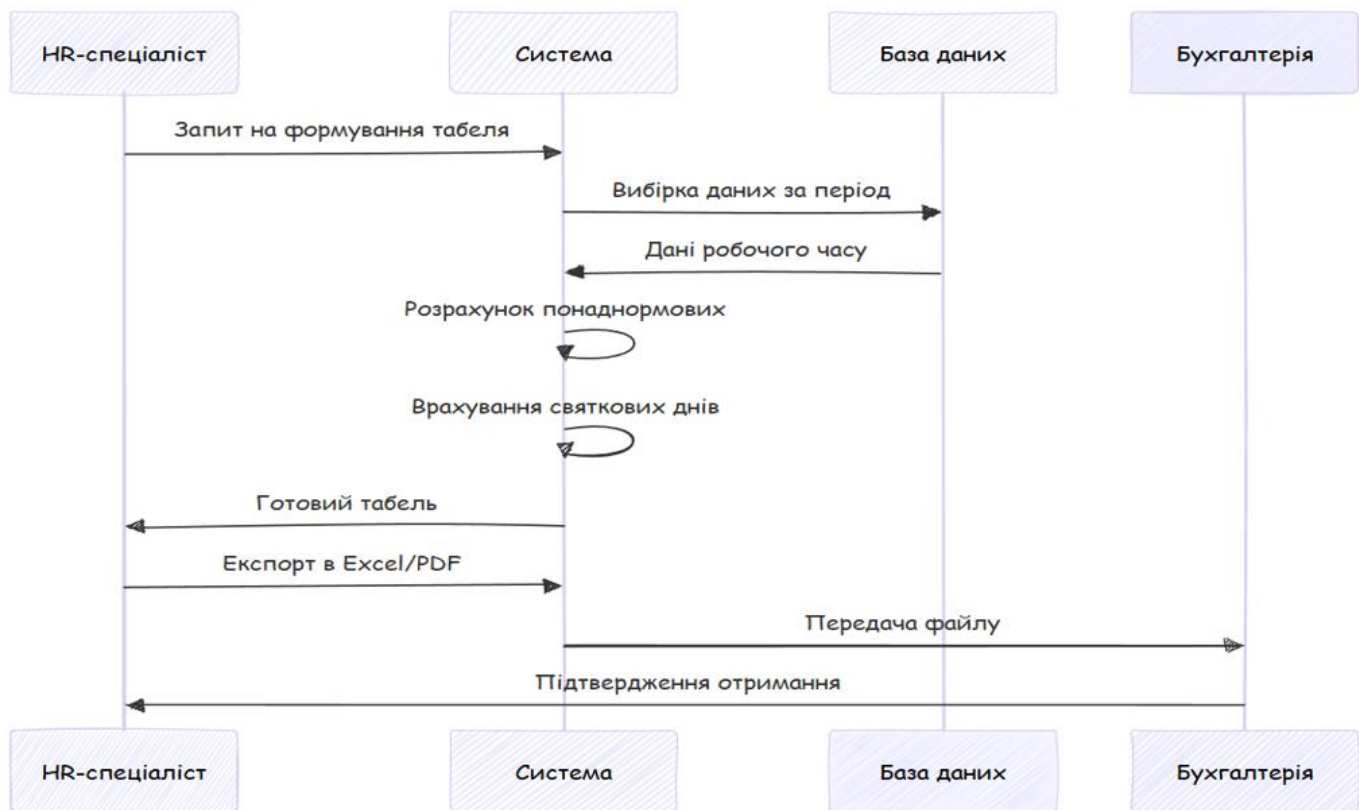


Рис. 5.5

4. Технічний адміністратор.

Людина, задача якої забезпечувати підтримку функціонування системи на технічному рівні і все що з цим пов'язано.

Типи операцій, функціональність технічного адміністратора зобразив на наступній схемі

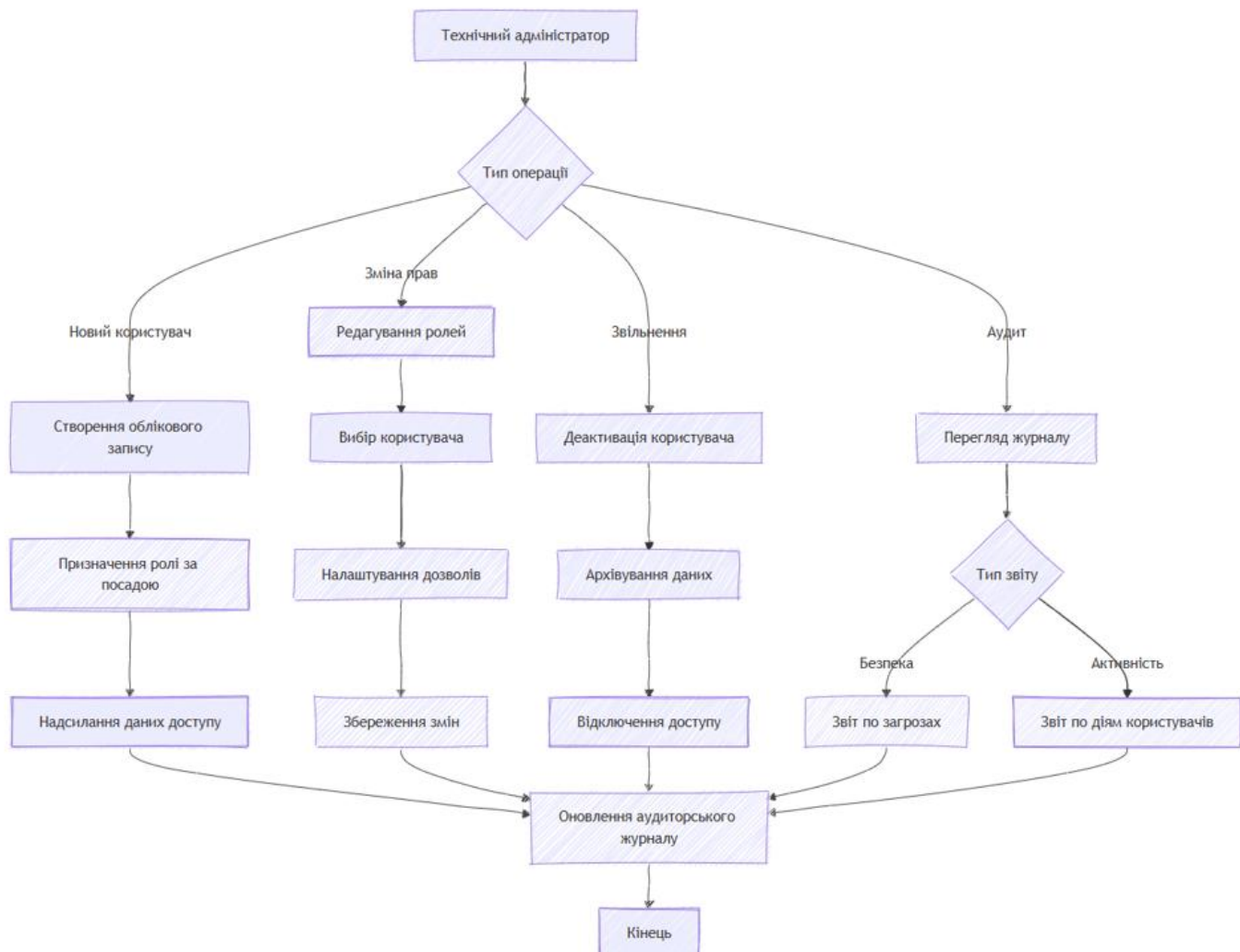


Рис. 5.6 Підтримка та функціонування системи.

Розділ 6.

Архітектура, залежності підсистем.

1. Рівні архітектури (логічна, фізична);

Логічна архітектура

Побудована буде за принципом багаторівневої **клієнт-серверної моделі**, буде мати чіткий розподіл обов'язків між компонентами системи. Архітектура складається з трьох основних логічних рівнів, кожен з яких має свої функціональні обов'язки та взаємодіє з суміжними рівнями через стандартизовані інтерфейси.

Рівень представлення.

Інтерфейс користувача та взаємодію з різними типами клієнтських пристроїв. Цей рівень забезпечує адаптивність до різних платформ і пристроїв, підтримуючи веб-браузери, мобільні пристрої та настільні додатки. Валідація введених даних йде на стороні клієнта, а також форматування інформації для відображення.

Тепер рівень бізнес-логіки

Він містить основні алгоритми та правила функціонування системи обліку робочого часу. А саме, тут реалізовані сценарії обробки заявок на відпустку, розрахунку робочих годин, формування звітів та контролю доступу користувачів. Цей рівень забезпечує дотримання бізнес-правил організації та інтеграцію з зовнішніми системами.

На останок рівень даних.

Забезпечує зберігання, пошук та управління інформацією системи. Включає реляційну базу даних для структурованих даних, файлове сховище для документів.

2 Фізична архітектура

Фізична архітектура це розміщення компонентів системи на апаратних ресурсах та мережева топологію взаємодії. Дана система розгортається в гібридній хмарно-локальній інфраструктурі.

Клієнтський рівень

Фізично представлений користувацькими пристроями: робочими станціями в офісах, мобільними телефонами та планшетами співробітників. Ці пристрої підключаються до системи через корпоративну мережу або інтернет з використанням захищених протоколів.

2. Клієнт-серверна архітектура

Протоколи взаємодії стандартизовані для забезпечення сумісності та можливості інтеграції. Між клієнтським рівнем та рівнем застосунків використовується **RESTful API** через **HTTPS** протокол. Взаємодія з рівнем даних здійснюється через оптимізовані **SQL**-з'єднання

3. Основні компоненти системи

Основне - веб-клієнт, який представляє собою веб-додаток (**PWA**), що працює в усіх браузерах. Реалізований з використанням **React** або **Vue.js** фреймворків, забезпечує адаптивний інтерфейс та офлайн-функціональність для критичних операцій. Підтримує різні способи аутентифікації, включаючи **QR**-коди та біометричну ідентифікацію.

Як допоміжне -настільний клієнт, який призначений для адміністраторів та HR-спеціалістів, що потребують розширеної функціональності. Розроблений як Electron-додаток або нативний Windows-додаток, забезпечує потужні інструменти звітності, масового імпорту даних та адміністрування системи.

3.2 Серверні компоненти;

1.Перше - API Gateway, виступає єдиною точкою входу для всіх клієнтських запитів, забезпечуючи аутентифікацію, авторизацію, *rate limiting* ;

2.Сервіс обліку робочого часу

реалізує основну бізнес-логіку системи: реєстрацію приходу/відходу, розрахунок робочих годин, контроль графіків роботи та обробку виняткових ситуацій. Включає систему тригерів для автоматичного сповіщення про порушення графіків.

3.Сервіс управління відпустками

обробляє життєвий цикл заявок на відпустку від подачі до затвердження, контролює баланс відпускних днів, інтегрується з календарними системами та забезпечує **workflow**-процеси погодження.

4.Сервіс звітності

генерує різноманітні звіти для різних категорій користувачів, підтримує експорт у множині форматів (**PDF, Excel, CSV**), забезпечує планування автоматичної генерації звітів та їх розсилку зацікавленим особам.

3 компонентів рівня даних;

PostgreSQL база даних служить основним сховищем структурованих даних системи. Налаштована реплікація **master-slave** для підвищення доступності.

Також, є **файловий сервер**, який зберігає документи, згенеровані звіти, завантажені файли, тощо. Реалізований як розподілена файлова система з автоматичним резервним копіюванням та версіонуванням файлів.

Взаємодія між компонентами системи організована за принципом слабого зв'язування через стандартизовані API та асинхронні повідомлення. Кожен мікросервіс має власну базу даних та відповідає за конкретну бізнес-функцію

Синхронна взаємодія

Використовується для операцій, що вимагають миттєвої відповіді: аутентифікація користувачів, реєстрація робочого часу, валідація даних. Реалізована через **RESTful API** з використанням **JSON** для обміну даними та **HTTP status codes** для індикації результатів операцій.

Асинхронна взаємодія також наявна, застосовується для довготривалих операцій та повідомлень між сервісами: генерація звітів, масові оновлення даних, сповіщення користувачів.

5. Обґрунтування вибору архітектури

Вибір трирівневої клієнт-серверної архітектури з мікросервісним підходом був вибраний через вимоги системи обліку робочого часу...

Система обліку робочого часу характеризується

специфічними патернами навантаження з виразними піками активності. Тобто, найвище навантаження спостерігається в години масового приходу та відходу співробітників (8:00-9:00 та 17:00-18:00), коли сотні або тисячі користувачів одночасно реєструють свій робочий час. У решті часу система працює з мінімальним навантаженням, обробляючи лише спорадичні запити на перегляд статистики та адміністративні операції.

Система повинна обслуговувати різноманітні сценарії використання від простої реєстрації часу до складної аналітики та звітності, для використання цих даних для аналітики\вирахування зп, тощо.

5. Переваги та недоліки

Модульна архітектура системи забезпечує гнучкість розвитку та підтримки, дозволяючи незалежно розробляти, тестувати й оновлювати окремі компоненти без ризику порушення роботи всієї системи. Розподілена структура підвищує доступність і відмовостійкість, адже збій одного модуля не призводить до повної недоступності — система зберігає працездатність критичних функцій. Оптимізація кожного рівня — бази даних, кешування та клієнтських застосунків — гарантує стабільну продуктивність навіть за високого навантаження. Централізоване управління безпекою через API Gateway забезпечує єдині політики автентифікації та авторизації, реалізуючи принцип «захисту в глибину».

З обмежень:

Висока складність розгортання та управління, оскільки вимагає зрілої **DevOps-практики** для моніторингу, логування і автоматизованого оновлення сервісів.

Система є критично залежною від стабільності мережевих з'єднань, тому мережеві затримки чи збої можуть впливати на продуктивність і якість користувацького досвіду...

Крім того, тестування взаємодії між численними сервісами є складнішим, ніж у монолітних системах, і потребує спеціалізованих інструментів для моделювання збоїв. Також підвищуються вимоги до компетенцій команди, тому необхідна участь фахівців із DevOps та мережевої безпеки.

Маштабування:

розподілена архітектура дозволяє масштабувати кожен компонент незалежно відповідно до навантаження.

Також, не варто забувати про інтеграційні можливості. Мікросервісна архітектура полегшує інтеграцію з існуючими корпоративними системами через стандартизовані API, що критично важливо для HR-системи, яка повинна обмінюватися даними з багатьма іншими системами організації

Архітектура системи дозволяє ефективно горизонтальне масштабування через додавання нових сервісів при зростанні навантаження. **API Gateway** зможе автоматично розподіляти запити між доступними інстансами мікросервісів, використовуючи алгоритми балансування навантаження.

Розділ 7.

Моделі даних.

Опис основних сутностей

Модель даних системи обліку робочого часу складається з семи основних сутностей, що відображають предметну область та забезпечують реалізацію функціональних вимог, визначених у Розділі 2. Для первинних ключів усіх сутностей використовується тип INTEGER з автоматичним інкрементуванням (AUTO_INCREMENT), що гарантує унікальність ідентифікаторів та ефективність виконання JOIN-операцій між таблицями.

Сутність 1 - Користувач (User).

Базова сутність системи, що зберігає облікові дані для автентифікації та авторизації всіх категорій користувачів. Поле username та email є унікальними в межах системи - це виключає дублювання облікових записів. Пароль зберігається виключно у вигляді хешу (password_hash), що відповідає вимогам безпеки NFR-03. Поле role визначає приналежність користувача до однієї з чотирьох ролей (employee, manager, hr, admin) і є основою для реалізації моделі RBAC. Поле is_active дозволяє деактивувати обліковий запис без видалення даних - наприклад, при звільненні співробітника.

Атрибут	Тип	Опис
user_id	INT, PK, AUTO_INCREMENT	Унікальний ідентифікатор
username	VARCHAR(50), UNIQUE	Логін користувача
password_hash	VARCHAR(255)	Хеш пароля
email	VARCHAR(100), UNIQUE	Електронна пошта
role	ENUM	Роль у системі
is_active	BOOLEAN	Статус активності
created_at	TIMESTAMP	Дата створення
updated_at	TIMESTAMP	Дата останнього оновлення

Сутність 2 - Працівник

Розширює сутність **User** додатковими кадровими даними, що стосуються конкретного співробітника організації. Зв'язок із **User** реалізований через зовнішній ключ **user_id** з унікальним обмеженням, що формує відношення 1:1. Поле **employee_number** - табельний номер - є унікальним ідентифікатором працівника у кадровому контексті та використовується при інтеграції з **HR/Payroll**-системами. Поле **department_id** визначає організаційну належність, а **hire_date** - дату початку трудових відносин.

Атрибут	Тип	Опис
employee_id	INT, PK, AUTO_INCREMENT	Унікальний ідентифікатор
user_id	INT, FK User	Зв'язок з обліковим записом
first_name	VARCHAR(100)	Ім'я
last_name	VARCHAR(100)	Прізвище
employee_number	VARCHAR(20), UNIQUE	Табельний номер
department_id	INT, FK Department	Відділ
position	VARCHAR(100)	Посада
hire_date	DATE	Дата прийняття на роботу

Сутність 3 - Відділ

Відображає організаційну структуру компанії та забезпечує групування працівників за підрозділами. Поле `manager_id` є зовнішнім ключем до таблиці `Employee` і вказує на відповідального керівника відділу. Це дозволяє системі визначати, хто саме має право затверджувати заявки та переглядати дані працівників конкретного підрозділу.

Атрибут	Тип	Опис
department_id	INT, PK, AUTO_INCREMENT	Унікальний ідентифікатор

name	VARCHAR(100)	Назва відділу
description	TEXT	Опис

Сутність 4 - Запис робочого часу

Центральна сутність системи, що фіксує факт присутності або відсутності працівника та є основою для формування табелів. Поля `check_in` і `check_out` зберігають мітки часу приходу та відходу відповідно до сценаріїв US-1 та US-2. Поля `break_start` / `break_end` дозволяють враховувати перерви при розрахунку чистого робочого часу. Поле `total_hours` є обчислюваним значенням (або обчислюється на рівні бізнес-логіки), що використовується у звітах. Поле `status` відображає тип дня (`present`, `absent`, `late`, `remote`), а `location` - місце виконання роботи (офіс, дистанційно тощо).

Атрибут	Тип	Опис
record_id	INT, PK, AUTO_INCREMENT	Унікальний ідентифікатор
employee_id	INT, FK → Employee	Працівник
work_date	DATE	Дата роботи
check_in	DATETIME	Час приходу
check_out	DATETIME	Час відходу

break_start	DATETIME	Початок перерви
break_end	DATETIME	Кінець перерви
total_hours	DECIMAL(5,2)	Загальний робочий час
status	ENUM	Статус дня
location	VARCHAR(100)	Місце роботи
notes	TEXT	Примітки

Сутність 7 - Аудиторський журнал (AuditLog)

Забезпечує повну простежуваність усіх змін у системі відповідно до вимог NFR-04 та функціональної вимоги FR-09. Кожен запис фіксує: хто (user_id) виконав дію (action), в якій таблиці (table_name) і для якого запису (record_id), а також зберігає старі та нові значення у форматі JSON - це дозволяє відновити стан даних на будь-який момент часу. Поле work_schedule_id забезпечує додатковий контекст для подій, пов'язаних із змінами у графіках.

Атрибут	Тип	Опис
og_id	INT, PK, AUTO_INCREMENT	Унікальний ідентифікатор
user_id	INT, FK → User	Ініціатор дії
action	VARCHAR(50)	Тип дії (CREATE, UPDATE, DELETE)

table_name	VARCHAR(50)	Таблиця, що змінювалась
record_id	INT	ID зміненого запису
old_values	JSON	Значення до зміни
new_values	JSON	Значення після зміни
timestamp	DATETIME	Час події

Зв'язки між сутностями

Один-до-одного (1:1):

User → Employee: кожен обліковий запис відповідає рівно одному профілю працівника (реалізовано через UNIQUE обмеження на user_id у таблиці Employee).

Один-до-багатьох (1:N):

Department до Employee: один відділ містить багатьох працівників.

Employee до TimeRecord: один працівник має багато записів робочого часу.

Employee до LeaveRequest: один працівник може подавати багато заявок на відпустку.

WorkSchedule до Employee: один графік може бути призначений багатьом працівникам.

Employee (manager) до LeaveRequest: один керівник може затверджувати багато заявок.

Розділ 8.

Вибір типу бази даних.

1. Вибір бази даних для системи та його обґрунтування.

Обрана БД:

-Реляційна база даних, PostgreSQL.

Обґрунтування вибору:

Є потреба зберігати дані в структурованому, табличному вигляді, тому що присутні багато списків(працівників, графіку роботи, аудиту, тощо), тобто це є оптимальним вибором через природу HR-даних та бізнес специфіки системи.

Система оперує високоструктурованими даними з передбачуваними та стабільними зв'язками: працівники належать до відділів, мають призначені графіки роботи, генерують записи робочого часу та подають заявки на відпустки. Ці зв'язки описуються чіткими бізнес-правилами та природно моделюються через первинні й зовнішні ключі реляційної моделі. Реляційна схема гарантує цілісність даних на рівні СУБД без додаткової логіки в застосунку.

Тобто, присутня структурованість даних та чіткі зв'язки. Облік робочого часу оперує високоструктурованими даними з чітко визначеними зв'язками між сутностями. В нас працівники належать до відділів, мають графіки роботи, створюють записи робочого часу та подають заявки на відпустки; ці зв'язки є стабільними, передбачуваними та описуються чіткими бізнес-правилами. Реляційна модель ідеально підходить для таких сценаріїв, тому що можна

моделювати ієрархії та зв'язки через первинні та зовнішні ключі.

Порівняння з альтернативою

Критерій	PostgreSQL (SQL)	MongoDB (NoSQL)
Структура даних	Чітка схема, таблиці	Гнучкі документи
Зв'язки між сутностями	Нативні JOIN, FK	Вбудовані документи або ручні посилання
Транзакційність	Повна ACID	Обмежена (multi-document)
Складні запити / звіти	SQL, віконні функції	Aggregation pipeline
JSON-підтримка	JSONB з індексуванням	Нативно

Плюс в **PostgreSQL** є Підтримка **JSON**, для зберігання аудиторських логів та гнучких налаштувань.

Отже, **postgreSQL** обрано як оптимальну СУБД для системи обліку робочого часу компанії **Farry**, оскільки вона забезпечує необхідну транзакційність, підтримує складну аналітику через **SQL**, гарантує цілісність зв'язків між сутностями та водночас надає гнучкість через підтримку **JSONB** для напівструктурованих даних аудиту.

Розділ 9.

Аналіз можливостей масштабування системи.

БД:

-Реляційна база даних, PostgreSQL.

Характер навантаження

Система обліку робочого часу компанії PhantomGroup характеризується нерівномірним навантаженням із чітко вираженими піковими періодами. Найвища активність припадає на масовий прихід та відхід співробітників (8:00–9:00 та 17:00–18:00), коли значна кількість користувачів одночасно реєструє свій робочий час. Протягом решти дня переважають поодинокі запити на перегляд статистики, адміністративні операції та фонові генерації звітів. Обрана трирівнева клієнт-серверна архітектура з мікросервісним підходом дозволяє масштабувати окремі компоненти незалежно, відповідно до характеру навантаження на кожен із них.

Горизонтальне масштабування

Горизонтальне масштабування є основною стратегією для даної системи, оскільки пікове навантаження має передбачуваний та циклічний характер. Ключові мікросервіси - реєстрації подій (check-in/check-out), автентифікації та збору логів - можуть масштабуватися горизонтально шляхом запуску додаткових інстансів за балансувальником навантаження (Load Balancer). API, описаний у Розділі 6, розподіляє вхідні запити між доступними інстансами,

забезпечуючи рівномірне завантаження. У пікові часові вікна кількість інстансів Time Tracking Service може автоматично збільшуватися (auto-scaling), а після завершення піку - зменшуватися, що оптимізує використання ресурсів.

Вертикальне масштабування

Вертикальне масштабування (збільшення обсягу оперативної пам'яті, кількості процесорних ядер, продуктивності дискової підсистеми) є доцільним переважно на ранніх етапах впровадження системи, коли кількість користувачів є відносно невеликою. Цей підхід простіший в реалізації та не потребує змін в архітектурі, однак має фізичну межу масштабування та не вирішує проблему пікового навантаження так ефективно, як горизонтальний підхід. У довгостроковій перспективі вертикальне масштабування розглядається як допоміжний захід, а не основна стратегія.

Масштабування бази даних

Масштабування рівня даних є окремим завданням, оскільки реляційні СУБД складніше масштабувати горизонтально порівняно з сервісами без стану. Для

PostgreSQL передбачено такі підходи:

Реплікація master-slave - основний вузол (master) обробляє операції запису, а репліки (slave) - читання. Це знижує навантаження на основний вузол при великій кількості запитів на перегляд звітів та статистики.

Connection pooling (наприклад, через PgBouncer) - зменшує кількість прямих з'єднань із БД, що критично важливо в пікові моменти масової реєстрації.

Отже, архітектура системи забезпечує можливість як горизонтального, так і вертикального масштабування. Пріоритетною стратегією є горизонтальне масштабування мікросервісів із auto-scaling у пікові години, реплікація БД для розподілу читання та кешування статичних довідникових даних.

Розділ 10.

Аналіз системи безпеки.

Доступ до системи забезпечується через багатфакторну автентифікацію: основний метод – логін та пароль

Увесь трафік між клієнтами та сервером шифрується за протоколом HTTPS/TLS. API Gateway є єдиною точкою входу та реалізує rate limiting для захисту від атак типу брут-форс та ДДос. Чутливі дані (паролі, персональні дані співробітників) зберігаються у зашифрованому вигляді; доступ до бази даних можливий лише з внутрішньої мережі серверного рівня — пряме з'єднання з клієнтського рівня виключено.

Аудит та контроль цілісності

Усі критичні дії в системі фіксуються в AuditLog (сутність описана в Розділі 7): зміни записів робочого часу, затвердження заявок, операції з обліковими записами. Журнал є незмінним для звичайних користувачів — редагування доступне лише технічному адміністратору. Це забезпечує повну простежуваність та можливість розслідування інцидентів.

Резервне копіювання бд. Виконується щоденно в автоматичному режимі з підтримкою інкрементальних бекапів. Копії зберігаються на окремому файловому сервері (Розділ 6) з версіонуванням. Процедура відновлення регулярно тестується для підтвердження цілісності резервних копій.

Розділ 11.

Перспективи розвитку системи.

Оскільки система наша не існує ізольовано, а повинна взаємодіяти з головною корпоративною системою та деякими зовнішніми сервісами, описаними раніше. API виступить як основний механізм зв'язку.

Реалізований він буде для системи обліку робочого часу як «RESTful» API архітектура, що як з назви можна побачити, базується на принципах REST (або передача репрезентативного стану). Буде використовувати протокол HTTPS для передачі даних. Вибір REST API обґрунтований універсальністю, простотою у використанні та доволі широкою підтримкою в сфері.

API Gateway буде як центральна точка інтеграції. Всі зовнішні запити до нашої системи будуть проходити через API Gateway, який буде виконувати функції єдиної точки входу. Тобто централізоване управління автентифікацією, авторизацією, rate limiting (це обмеження частоти запитів).

Основні принципи RESTful будуть включати наступне. Stateless природа. Кожен HTTP запит буде містити всю необхідну інформацію для його обробки. Реалізована через jwt (json web токени), які клієнт отримує після успішної авторизації та включає в заголовки всіх наступних запитів. А також використання стандартних HTTP методів.

Ієрархічна структура ресурсів. Ендпоінти API будуть організовані ієрархічно. Наприклад, для отримання записів робочого часу конкретного працівника можна використати умовний шлях `/api/v1/employees/{employeeId}/`

timerecords, що явно вказує на зв'язок між працівником та його записами часу.

Ну й JSON як основний формат даних. Всі запити та відповіді API будуть використовувати JSON формат, що дасть читабельність, компактність та широку підтримку в різних мовах програмування.

2. Моделювання потоків даних між системами.

Система обліку робочого часу буде взаємодіяти з множиною зовнішніх та внутрішніх компонентів через чітко визначені потоки даних. Ці потоки розділяються на кілька категорій: синхронні операції користувачів (реєстрація часу, перегляд статистики), асинхронні інтеграції з корпоративними системами (це синхронізація даних з ERP, відправлення інформації до платіжної системи, та системні процеси, як генерація звітів, очищення кешу). Взаємодія відбувається між чотирма основними шарами: клієнтські додатки, API Gateway, мікросервіси, шар даних. Діаграму взаємодії цих шарів представив на рис. 7.1

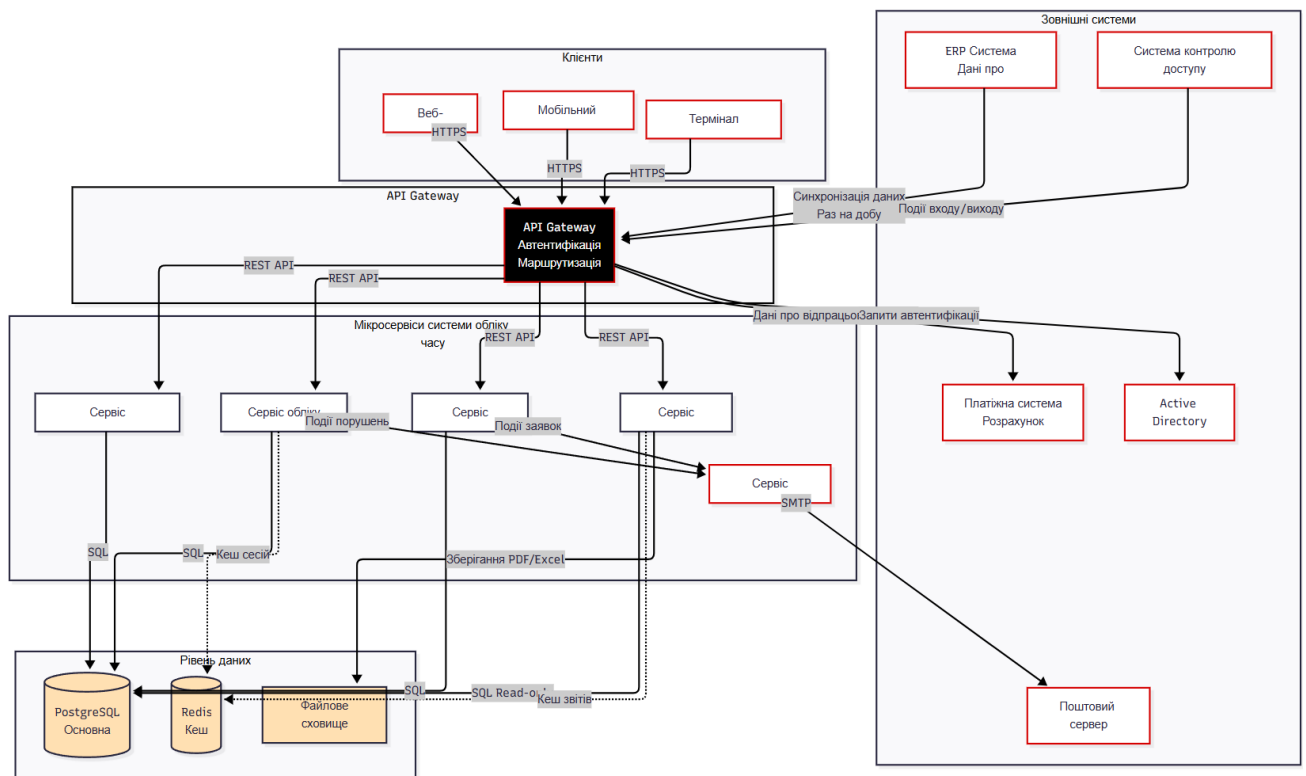


Рис. 11,1

Клієнтський шар.

Система має три типи клієнтів: веб-додаток, мобільні додатки Android/iOS і термінали реєстрації. Веб-інтерфейс забезпечує повний доступ до функцій, мобільні - дозволяють фіксувати робочий час і переглядати статистику, а термінали підтримують швидку ідентифікацію через QR-коди чи біометрію. Усі клієнти працюють через протокол HTTPS. Далі шар API Gateway. Точка входу до системи. Перевіряє автентичність запитів, маршрутизує їх до потрібних мікросервісів, контролює навантаження, кешує відповіді.

Мікросервісний шар. Це п'ять мікросервісів: автентифікація - керування користувачам. Облік часу - реєстрація робочих змін, розрахунок годин і виявлення порушень. Відпустки - обробка заявок, контроль балансу днів. Сповіщення - централізована відправка email, push та SMS.

Шар зберігання даних.

Два сховища:

PostgreSQL - основна база для структурованих даних.

Файлове сховище - для всіляких звітів, документів і архівів.

Redis - кеш для тимчасових даних і прискорення запитів, не є сховищем, але є обов'язковим, тому виділив на схемі.

Також система синхронізується з корпоративною ERP, отримуючи оновлені дані про працівників. Зобразив на Рис 7.2.

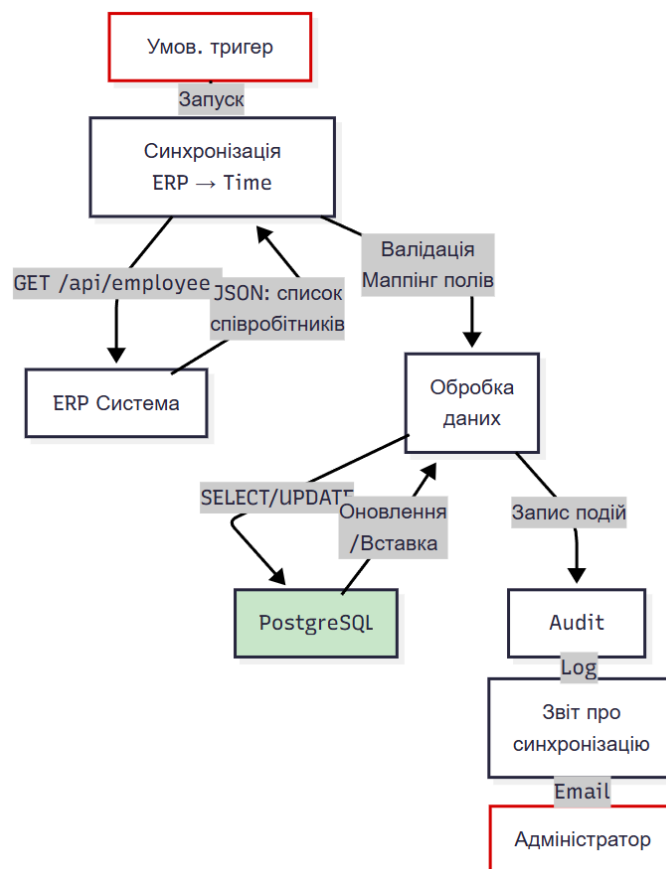


Рис. 11.2

У поточній реалізації система зосереджена на базовому обліку робочого часу та управлінні відпустками. У перспективі можливе розширення функціоналу: впровадження модуля планування завдань (task tracking) із прив'язкою витраченого часу до конкретних проєктів, автоматичний розрахунок заробітної плати безпосередньо всередині системи без необхідності передачі даних у зовнішній Payroll-модуль, а також розширена аналітика продуктивності підрозділів на основі накопичених даних.

Інтеграція з іншими підсистемами

Система спроектована як автономний модуль з відкритими інтеграційними інтерфейсами (Розділ 6), що закладає основу для подальших інтеграцій. Серед пріоритетних напрямів — глибша інтеграція з корпоративними ERP/HRM-системами, підключення біометричних пристроїв і турнікетів для автоматичної реєстрації без участі користувача, а також синхронізація з корпоративними календарями (Google Calendar, Microsoft Outlook) для відображення графіків і відпусток.

Розділ 12.

Методологія розробки.

1.Обрана методологія: ітеративна розробка з елементами Agile з використанням елементів підходу Scrum.

Вважаю, що для системи обліку робочого часу слід використати гнучку методологію Agile з використанням елементів підходу Scrum. Чому? Тому що є потреба поступового уточнення вимог, швидко реагувати на зміну пріоритетів замовника. В якості даного проєкту команда розробки розглядається як невелика кросфункціональна група (я, як студент розробник, аналітик і частково тестувальник, викладач у ролі замовника/стейкхолдера).

Це узгоджується з принципами Agile, де важливі короткі ітерації, постійний зворотний зв'язок і поступове нарощування функціональності. Тобто проєкт розбито на короткі ітерації (типу «спринтів»), кожна з яких відповідає окремому блоку робіт. Є узгоджена мета (закрити певний розділ системного аналізу), короткі цикли планування (постановка завдань на практичних), та регулярний review в якості перевірки замовником(викладачем) проміжних результатів.

Структура системи та обрана мікросервісна логіка також узгоджується з Agile. Кожен сервіс (облік часу, управління відпустками, звітність, аутентифікація, аудит) можна розглядати як окремий елемент, який розробляється и тестується незалежно.

Структура ітерацій (спринтів):

Проект буде розбито на короткі ітерації, кожна з яких відповідає окремому блоку робіт із системного аналізу та проєктування:

Ітерація 1 - визначення предметної області, збір та аналіз вимог (Розділи 1-2).

Ітерація 2 - визначення користувачів, ролей та user stories (Розділи 3-4).

Ітерація 3 - UML-моделювання та діаграми послідовності (Розділ 5).

Ітерація 4 - архітектура системи та модель даних (Розділи 6-7).

Ітерація 5 - вибір СУБД, масштабування, безпека (Розділи 8-10).

Ітерація 6 - перспективи розвитку, методологія, документування (Розділи 11-13).

Розділ 13.

Документація проекту перед фінальною збіркою.

Тідсумкова робота оформлена як повноцінна проєктна документація інформаційної системи обліку робочого часу для компанії Phantom Group. Документ охоплює повний цикл системного аналізу та проєктування - від визначення предметної області до обґрунтування архітектурних рішень і перспектив розвитку.

Мета роботи - виконання системного аналізу та проєктування інформаційної системи обліку робочого часу, яка забезпечить структурований збір даних про присутність співробітників, підтримує рольову модель доступу та інтегрується з існуючою певною інфраструктурою компанії.

Завдання роботи:

Завданням роботи є продемонструвати здатність спроектувати інформаційну систему повного циклу - від аналізу вимог і моделювання бізнес-процесів до побудови архітектури, інтеграції компонентів і документування результатів.

На даний момент документ складається з 16 основних розділів та додатків:

Розділ 1 - визначення предметної системи, обґрунтування вибору та актуальності автоматизації обліку робочого часу.

Розділ 2 - збір та аналіз вимог: специфікація функціональних вимог (FR-01-FR-10) та нефункціональних вимог), опис бізнес-процесів AS-IS.

Розділ 3 - визначення користувачів системи: чотири ролі (працівник, лінійний керівник, HR-спеціаліст, технічний адміністратор), матриця доступу на основі RBAC.

Розділ 4 - функціональні сценарії (user stories) для кожної ролі з критеріями прийняття.

Розділ 5 - діаграми послідовності (UML Sequence Diagrams) для сценаріїв check-in та процесу коригування/затвердження робочого часу.

Розділ 6 - логічна та фізична архітектура системи, опис мікросервісів, протоколів взаємодії та зовнішніх інтеграцій.

Розділ 7 - модель даних: опис семи сутностей, їх атрибутів, зв'язків та ER-діаграма.

Розділ 8 - обґрунтування вибору PostgreSQL як реляційної СУБД, порівняння з NoSQL-альтернативою.

Розділ 9 - аналіз можливостей горизонтального та вертикального масштабування, кешування та резервування.

Розділ 10 - аналіз системи безпеки: автентифікація, авторизація, захист даних, аудит та резервне копіювання.

Розділ 11 - перспективи розвитку системи: розширення функціоналу, нові інтеграції, мобільний клієнт, хмарна інфраструктура.

Розділ 12 - обґрунтування вибору методології Agile, структура ітерацій проєкту.

Розділ 13 - документування результатів.

Розділ 14 - Цикли системи.

Розділ 15 – Артефакти системи.

Розділ 16 – Точки впливу.

В документі є графічні матеріали, виконані в UML-нотації та у вигляді схем:

Мал. 3.1 - схематичне представлення різних категорій користувачів.

Мал. 5.1 - діаграма станів працівника (State Machine Diagram).

Мал. 5.2- блок-схема сценарію перегляду статистики.

Мал. 5.3 - діаграма станів заявки на відпустку.

Мал. 5.4 - Mind map функцій лінійного керівника.

Мал. 5.5 - Sequence Diagram: check-in / початок зміни.

Мал. 5.6 - Sequence Diagram: коригування часу та затвердження керівником.

Мал. 11.1 - взаємодія шарів системи.

Мал. 11.2 синхронізація з корпоративною ERP.

Також ER-діаграма моделі даних системи.

Висновок про відповідність вимогам та якість проєктного рішення.

Спроектована система обліку робочого часу відповідає визначеним функціональним вимогам (FR-01-FR-10) та нефункціональним вимогам (NFR-01-NFR-06). Архітектурне рішення на основі трирівневої клієнт-серверної моделі з мікросервісним підходом забезпечує масштабованість, модульність та можливість незалежного розвитку окремих компонентів. БД PostgreSQL як СУБД дасть структуру HR-даних й адміністраторів. Модель безпеки буде виконана на основі RBAC та аудит-логу, щоб побачити усі зміни системи.

Розділ 14.

Визначення циклів у системі

Відповідно до принципів системного мислення, моя інформаційна система - це не статичний набір модулів, а динамічна структура з петлями зворотного зв'язку (Feedback Loops), які визначають її поведінку в часі. Система обліку робочого часу містить кілька таких петель, що впливають на якість даних, поведінку користувачів та стабільність роботи. В системі присутні наступні цикли:

Підсилюючі цикли

Цикл R1 - Довіра до системи:

Чим точніше система фіксує робочий час -> тим більше співробітників довіряють її даним -> тим сумлінніше вони реєструють check in / check out -> тим точнішими стають дані. Це підсилюючий цикл зростання якості: система покращується разом із дисципліною користувачів.

Цикл R2 - Накопичення помилок (негативний підсилюючий):

Якщо інтерфейс незручний або незрозумілий -> користувачі обходять систему або вносять некоректні дані -> якість даних падає -> звіти стають ненадійними -> довіра до системи знижується -> користувачі ще менше мотивовані працювати з нею коректно. Поганий інтерфейс змушує людей обходити систему.

Цикл R3 - Зростання навантаження:

Зі зростанням кількості співробітників компанії PhantomGroup -> зростає обсяг записів у TimeRecord ->

зростає навантаження на базу даних -> при відсутності масштабування сповільнюється відповідь системи -> знижується задоволеність користувачів. Складність зростає нелінійно.

Балансуючі цикли

Цикл В1 - Контроль якості даних:

Аудиторський журнал (AuditLog) фіксує всі зміни -> HR-спеціаліст або керівник виявляє аномалії -> вносяться корекції -> якість даних відновлюється до прийнятного рівня. Це балансує цикл, що утримує систему в стані достовірності даних.

Цикл В2 - Управління заявками на відпустку:

Співробітник подає заявку -> керівник отримує сповіщення та розглядає -> затверджує або відхиляє -> статус заявки змінюється -> черга очікуючих заявок скорочується. Система прагне до нульової черги необроблених заявок - це класичний балансує цикл управління станом.

Цикл В3 - Авто-масштабування навантаження:

У пікові години (8:00-9:00, 17:00-18:00) зростає кількість запитів -> система виявляє перевантаження -> авто сканування запускає додаткові інстанси трекінгового сервісу часу -> навантаження розподіляється -> час відповіді повертається до норми.

Розділ 15.

Визначення артефактів системи

Артефакти - по суті інформаційні сутності. Артефакт є інструментом перевірки та передачі знань між учасниками проєкту. Моя система обліку робочого часу має артефакти на кожному етапі свого життєвого циклу.

Входять сюди спочатку вимоги.

Специфікація функціональних (FR-01-FR-10) та нефункціональних вимог (NFR-01-NFR-06), Вимога відповідає на питання "ЩО", а не "ЯК". Це важливо, тому що змішування цих рівнів робить ТЗ крихким.

Концепція

Визначає межі системи, потоки даних та принципи безпеки - міст між побажаннями замовника та інженерною реалізацією. Концепція системи відображена в Розділі 6 (архітектура) та Розділі 3 (ролі та доступ).

Технічне завдання (ТЗ)

Фіксує цілі, межі та критерії успіху проєкту. Захищає від діалогу "а ми думали, що..." між розробником та замовником, знімаючи двозначність ще до початку реалізації.

Моделі (UML-діаграми)

Діаграми послідовності (Розділ 5), ER-діаграма (Розділ 7), діаграми станів - структурні та поведінкові артефакти системи. Моделі дешевше виправляти ніж код, тому вони є

обов'язковим проміжним артефактом між вимогами та реалізацією.

Архітектура

Логічна та фізична архітектура (Розділ 6) є артефактом, що захищає від ефекту доміно: зміни в одному мікросервісі не ламають інші завдяки чітким межам відповідальності та фіксованим інтерфейсам взаємодії.

Звісно код, як кінцевий артефакт

Код є результатом усіх попередніх артефактів - вимог, концепції, моделей та архітектури. Система без документації це знання в головах, які зникають разом із людьми, що її створювали.

Відповідність системи артефактам відобразив в табл. 15.1:

Артефакт	Де відображено в проєкті
Вимоги (FR/NFR)	Розділ 2
Концепція	Розділи 3, 6
ТЗ / межі системи	Розділ 1
UML-моделі	Розділи 5, 7
Архітектура	Розділ 6
Модель даних	Розділ 7
Документація	Розділ 13

Розділ 16.

Точки впливу на систему.

Точки впливу це місця в системі, де відносно невелике втручання дає непропорційно великий ефект на поведінку всієї системи. Ключові питання при їх визначенні: де виникають дані, хто відповідає за помилки і де справжні вузькі місця.

Модель даних є точкою з найвищою значущістю - вона є фундаментом системи, і помилка на цьому рівні створює довгий хвіст проблем: звіти стають недостовірними, історія губиться, інтеграції з зовнішніми системами ламаються. Зміна схеми даних на пізньому етапі коштує надзвичайно дорого, тому правильне рішення, закладене в Розділі 7, визначає якість усієї системи.

Інтерфейс реєстрації є не лише засобом взаємодії, а механізмом безпеки та єдиною точкою входу всіх первинних даних у систему. Якщо процес реєстрації незручний - користувачі обходять систему або вносять некоректні дані, Покращення UX цієї точки впливає на усі звіти.

Система ролей та доступу (RBAC) визначає хто і що може робити в системі. Це точка контролю відповідальності: зміна правил доступу миттєво змінює поведінку всіх користувачів без зміни коду бізнес-логіки. Архітектура - це вибір місця для правил, і RBAC є саме таким місцем.

Аудиторський журнал є точкою спостереження за всією системою: фіксує відхилення, дозволяє виявляти аномалії та запускати балансуєчий цикл відновлення якості даних.

На останок **API**, як єдина точка входу для всіх запитів. Забезпечує контроль навантаження, безпеки та маршрутизації одночасно. Зміна правил на цьому рівні, як автентифікація, балансування - впливає на всі компоненти системи без втручання у внутрішню логіку окремих сервісів.

Кінцевий висновок і подальший можливий розвиток системи обліку робочого часу.

Тож, проведено повний цикл системного аналізу до проєктування інформаційної системи, а саме системи обліку робочого часу. Було формалізовано вимоги й опис користувачів з виконанням UML моделювання, проєктування архітектури, вибір бази даних, аналіз масштабування, взаємодія систем та вибір методології для розробки системи обліку робочого часу. Запропоноване рішення базується на трирівневій клієнт серверній архітектурі з елементами мікросервісного підходу, реляційній СУБД PostgreSQL та рольовій моделі. Системний аналіз підтвердив, що автоматизація обліку робочого часу усуває ключові проблеми ручного ведення табелів.

Щодо подальшого розвитку реалізованого рішення системи обліку робочого часу, пропонується розробка повноцінного прототипу веб - та мобільного інтерфейсів, впровадження аналітичних дашбордів і модулів для аналізу навантаження на систему. А також, можливо, модуля для аналізу понаднормових годин і розширення інтеграцій із зовнішніми системами (ERP) з подальшим посиленням безпеки, якщо система буде розширюватись та масштабуватись.